Aula 10 — KNN e Árvores de Classificação

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §8.3–8.6; ISLP §4.7.6 e cap. 8

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- adaptar o **KNN** para classificação (voto da maioria) e estimar $\mathbb{P}(Y = c \mid \boldsymbol{x})$ pelas proporções entre vizinhos;
- construir uma **árvore de classificação** e explicar a predição pela *moda*;
- definir os critérios de impureza **Gini** e **entropia** e justificar por que se preferem à taxa de erro;
- usar **florestas** e **boosting** para classificação (voto, m, AdaBoost);
- **comparar** os classificadores vistos no curso quanto a fronteira, interpretabilidade e desempenho.

Em sala

Aula que fecha a classificação reaproveitando dois métodos não paramétricos vistos em regressão (KNN, Aula 04; árvores, Aula 05), agora para classificar. Boa notícia: a estrutura é a mesma; muda só o "resumo local" (média → moda) e o critério de divisão (EQM → impureza).

1 KNN para classificação

Adaptação direta da regressão KNN (Aula 04): para classificar $\boldsymbol{x}$, olhe seus k vizinhos mais próximos e tome a **classe mais frequente** (voto da maioria):

$$\widehat{g}(\boldsymbol{x}) = \text{moda}\{Y_i : i \in \mathcal{N}_{\boldsymbol{x}}^{(k)}\}, \tag{1}$$

em que $\mathcal{N}_{\boldsymbol{x}}^{(k)}$ é o conjunto dos índices das k observações de treino mais próximas de $\boldsymbol{x}$. Equivalentemente, é um classificador *plug-in* (Aula 07): estima-se

$$\widehat{\mathbb{P}}(Y = c \mid \boldsymbol{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\boldsymbol{x}}^{(k)}} \mathbf{1}\{Y_i = c\}$$

(a proporção de vizinhos da classe c) e toma-se o $\arg\max_c$. Essa estimativa está, por construção, em $[0, 1]$.

Ideia central

k controla o balanço viés–variância, como na regressão: $k = 1$ dá fronteira muito "recortada" (baixo viés, alta variância); k grande dá fronteira suave (alto viés, baixa variância). Escolha por validação cruzada (Aula 03).

Atenção

Valem os mesmos cuidados da Aula 04: *padronize* as covariáveis (KNN usa distância) e lembre da *maldição da dimensionalidade* — KNN não seleciona variáveis e degrada com muitas covariáveis irrelevantes. Em binário, use k *ímpar* para evitar empates.

2 Árvores de classificação

São o análogo das árvores de regressão (Aula 05): partição recursiva do espaço de covariáveis em regiões $R_1, \ldots, R_J$. Duas adaptações:

2.1 Predição: a moda

Numa folha R_k, prediz-se a **classe majoritária** (em vez da média):

$$\widehat{g}(\boldsymbol{x}) = \text{moda}\{Y_i : \boldsymbol{X}_i \in R_k\}, \qquad \boldsymbol{x} \in R_k, \tag{2}$$

e $\widehat{\mathbb{P}}(Y = c \mid \boldsymbol{x}) = \widehat{p}_{R_k,c}$, a proporção da classe c na folha.

2.2 Critério de divisão: impureza

Em sala

Pergunta para a turma, antes das fórmulas: *"como uma árvore decide a melhor pergunta a fazer em cada nó quando o alvo é uma classe?"*

O EQM não faz sentido com Y qualitativo. Em cada região R, seja $\widehat{p}_{R,c}$ a proporção da classe c. Buscam-se divisões que tornem as folhas **puras** (idealmente, uma só classe por folha). Mede-se a impureza por:

$$\text{Índice de Gini:} \quad G(R) = \sum_{c \in \mathcal{C}} \widehat{p}_{R,c}\,(1 - \widehat{p}_{R,c}), \tag{3}$$

$$\text{Entropia:} \quad D(R) = -\sum_{c \in \mathcal{C}} \widehat{p}_{R,c} \log \widehat{p}_{R,c}. \tag{4}$$

Ambas são *mínimas* (zero) quando a folha é pura (algum $\widehat{p}_{R,c} = 1$) e *máximas* quando as classes estão equilibradas. A cada passo escolhe-se a divisão que mais reduz a impureza (ponderada pelo tamanho das folhas).

Observação 2.1 (Por que não usar a taxa de erro?)**.** A taxa de erro de classificação $1 - \max_c \widehat{p}_{R,c}$ também mede impureza, mas é **insensível** a mudanças nas proporções que não alterem a classe majoritária. Gini e entropia são *diferenciáveis* e respondem a qualquer melhora na pureza — por isso fazem divisões melhores. (A taxa de erro é boa para a *poda* final.)

Exemplo 2.2 (Escolha de divisão por Gini)**.** Para decidir entre dividir por "Cardiologia" ou por ">100 leitos" (predição: hospital tem tomógrafo), calcula-se o Gini ponderado de cada partição e escolhe-se a de *menor* valor (maior pureza). No exemplo do [AME], Gini(Leitos)$\approx 0{,}326 <$ Gini(Cardiologia)$\approx 0{,}472$: divide-se por leitos.

A construção segue as duas etapas da Aula 05: *cresce-se* uma árvore grande e depois *poda-se* por custo–complexidade (α por CV). As mesmas vantagens valem: interpretabilidade, categóricas nativas, seleção automática de variáveis e captura de interações.

3 *Ensembles* para classificação

Toda a Aula 05 se transfere; as árvores individuais agora classificam:

Bagging / Florestas treina B árvores em amostras bootstrap e agrega por **voto da maioria** (ou média das probabilidades). Florestas descorrelacionam sorteando m variáveis por nó; regra empírica em classificação: $m \approx \sqrt{p}$. Robustas a B; OOB e importância de variáveis "de graça".

Boosting ajuste sequencial. A variante clássica para classificação é o **AdaBoost**: a cada rodada, reponderam-se as observações dando mais peso às mal classificadas, e o classificador final é um voto ponderado dos aprendizes fracos. B grande pode superajustar (use *early stopping*); XGBoost é a implementação de referência.

4 Comparando os classificadores do curso

Método	Fronteira	Interpretab.	Observações
Logística	linear (no *log-odds*)	alta (coefs.)	dá probabilidades; robusta
LDA / QDA	linear / quadrática	média	ótimos sob gaussianidade
Bayes ingênuo	depende	média	forte com texto; indep. condicional
KNN	muito flexível, local	baixa	sensível a escala e dimensão
Árvore	"escada" (eixos)	**altíssima**	instável sozinha
Florestas / Boosting	muito flexível	baixa	topo de desempenho em dados tabulares
SVM (kernel)	linear ou não linear	baixa	ótima com classes separadas

Ideia central

Não há método universalmente melhor ("não existe almoço grátis"). A escolha depende da *estrutura* dos dados e do *objetivo*: precisa de probabilidades calibradas? interpretabilidade? muitas covariáveis irrelevantes? fronteira não linear? Compare alguns candidatos por validação cruzada e use as métricas adequadas (Aula 08).

Em sala

Use os notebooks `Aula prática 10`, `Comparação entre classificadores paramétricos` e o `Exemplo - KNN e árvores em dados artificiais` para *visualizar* as fronteiras de decisão lado a lado. É o melhor jeito de fixar a intuição de "flexível vs. rígido" que percorreu o curso inteiro.

Em sala: o item "probabilidades calibradas" não é preferência

A §7 da `Aula prática 10` refaz a tabela numa base de classe rara (10% de positivos), e vale mostrar as duas medições:

- a **acurácia** varia de 0,8989 a 0,9072 entre os sete métodos, e quatro empatam

exatamente com quem responde "não" para todo mundo, porque nunca preveem positivo. A AP varia de 0,119 a 0,453 na mesma tabela;
- com o corte de custo 1:20 da Aula 08 — 0,0476 — a árvore podada e o AdaBoost mandam investigar **todos os mil por mil**, porque as probabilidades deles quase nunca descem abaixo de 0,05. A logística manda 410 e acha 82 dos 101 positivos; o SVM manda 493 e acha 89.

A moral a extrair no quadro: o corte ótimo é ótimo *para probabilidade honesta*. Pergunte à turma o que fazer com um modelo que ordena bem e calibra mal — a resposta é da Aula 08, e é calibrar, não trocar de modelo.

5 Prática em Python

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

knn = make_pipeline(StandardScaler(),
                    KNeighborsClassifier(n_neighbors=11)) # k por CV
arv = DecisionTreeClassifier(criterion="gini", ccp_alpha=0.0) # ou "entropy"
rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                            oob_score=True, random_state=0)
ada = AdaBoostClassifier(n_estimators=200, learning_rate=0.5)

for nome, modelo in [("KNN", knn), ("arvore", arv), ("RF", rf), ("Ada", ada)]:
    modelo.fit(X_tr, y_tr)
    print(nome, modelo.score(X_te, y_te)) # use tambem as metricas da Aula 08
```

Resumo

- **KNN classificação** (1): voto da maioria; estima $\mathbb{P}(Y = c \mid \boldsymbol{x})$ por proporções; k por CV; padronize.
- **Árvore de classificação**: prediz a moda (2); divide por **Gini** (3) ou **entropia** (4) (melhores que a taxa de erro); cresce e poda como na Aula 05.
- **Florestas** ($m \approx \sqrt{p}$) e **boosting/AdaBoost** agregam árvores por voto — topo de desempenho em dados tabulares.
- Não há classificador universalmente melhor: escolha pela estrutura/objetivo e compare por CV com as métricas da Aula 08.

Leitura recomendada. [AME] §8.3 (árvores de classificação, índice de Gini), §8.4–8.5 (*bagging*, florestas, *boosting*), §8.6 (KNN); [ISLP] §4.7.6 (KNN) e Capítulo 8 (árvores e *ensembles*, incluindo critérios de impureza e *boosting*).